Syed Tasneem
100005567

RAG Pipeline Evaluation Report

SRH University berlin

Large Language Models, Prompting, and Agentic AI

Syed Tasneem
100005567

1. What I was trying to build

This report covers a small retrieval-augmented generation pipeline built over five PDFs: a report on a farmer cooperative marketplace platform, and four short AI research papers covering pronunciation correction in text-to-speech, cross-attention interpretability in TTS, and LLM compliance behaviour under many-shot demonstrations. Together the five documents run to roughly fifty pages once you count references and figures. I picked this mix on purpose rather than five similar documents, because I wanted questions that actually needed retrieval and not just world knowledge the model already had memorised.

I wrote twenty test questions before writing any pipeline code. Some are pure factual lookups (a percentage, a memory type, a training stage), some are multi-hop questions that need two passages from the same document stitched together, and a handful ask the model to compare or summarise across documents rather than quote a single fact. The full list is in the questions.py section of the repository; the first fourteen are reproduced in the evaluation table below because that is as far as one full run got before the free-tier API started throttling requests, which I discuss in the failure section.

2. Baseline pipeline and why each piece was chosen

The pipeline is built with LangChain, loading PDFs with PyPDFLoader, chunking with RecursiveCharacterTextSplitter, embedding with a local Sentence-Transformers model, and storing vectors in FAISS. Generation is handled by Llama 3.3 70B through the Groq API, mainly because Groq's inference is fast and free to use for a project of this size, and 70B parameters is large enough to follow an instruction like 'answer only from context' reliably.

I chose sentence-transformers/all-MiniLM-L6-v2 for embeddings instead of an OpenAI or Cohere embedding model for a simple reason: it runs locally, costs nothing, and 384 dimensions is more than enough separation for a fifty-page corpus. A bigger embedding model would help more once the corpus grows into the thousands of documents, but for five documents the retrieval quality bottleneck was almost never the embedding model it was chunk boundaries and which similarity metric pulled back the right passage.

FAISS was chosen over Chroma mainly for speed of setup: it needs no server process, indexes in memory, and is fast enough for a corpus this size. That trade-off would need revisiting at scale, which I return to in the last section.

3. Chunking and retrieval experiments

I tested two chunking strategies. Strategy A uses small, 500-character chunks with 50-character overlap, which gives 415 chunks across the corpus. Strategy B uses larger, 1500-character chunks with 200-character overlap and a wider set of separators (paragraph, line, sentence, word), giving 155 chunks. The intuition was that small chunks would be more precise for single-fact lookups but would lose surrounding context for anything that needed two sentences to make sense, while large chunks would carry more context but dilute the embedding with less relevant text.

On top of each chunking strategy I tested two retrieval methods: plain cosine similarity search with k=3, and Maximal Marginal Relevance (MMR) with k=3 and fetch_k=10, which re-ranks the initial candidate pool to reduce redundancy between the returned chunks. That gives four configurations in total, run against every one of the fourteen completed questions, plus a fifth 'no retrieval' condition where the same question goes straight to the LLM with no context at all, as a sanity check on how much retrieval is actually helping.

The results were not what I expected going in. I assumed MMR would consistently help by avoiding near-duplicate chunks, but on the small-chunk index it sometimes hurt: for question 11, asking which model was most robust to many-shot demonstrations, both small-chunk configurations returned 'not enough information in the context', while both large-chunk configurations correctly named GPT-OSS-20B. The large chunks were carrying the comparison sentence in one piece; the small chunks had split the model name from the robustness claim across two separate chunks, and neither cosine similarity nor MMR happened to retrieve both halves together. That is a chunking failure, not a retrieval failure, and no amount of re-ranking fixes a chunk boundary that has already cut a sentence in half.

The clearest overall pattern is that large chunks with MMR (configuration 4) gave the most complete answers on the harder multi-hop questions, at the cost of occasionally pulling in tangential passages that made an answer longer and slightly less focused than it needed to be. Small chunks with cosine similarity were the fastest to a correct short answer when the fact lived entirely inside one sentence question 7 and question 8, both single numeric facts, scored perfectly across all four configurations, which tells me chunking strategy barely matters once the retrieval problem itself is easy.

4. Evaluation results

Scores are out of 5, judged by reading each answer against the source PDF and marking factual accuracy, completeness, and whether the answer stayed grounded in the retrieved context rather than guessing. A score of 0 means the question was not reached before the run stopped.

Q#	Question topic	Cfg 1: Small+Cosine	Cfg 2: Small+MMR	Cfg 3: Large+Cosine	Cfg 4: Large+MMR
1	Farmer Connect main objective	4	3	4	4
2	Allocation algorithm (FIFO)	5	5	5	2
3	Four user roles	5	5	5	5
4	Support for non-smartphone farmers	2	4	5	5
5	FlowEdit main problem	4	4	4	2
6	FlowEdit memory type	5	5	5	4
7	PER reduction percentage	5	5	5	5
8	Correction time on single GPU	5	5	5	4
9	Three compliance hypotheses	4	4	5	5
10	Training stage preventing harmful compliance	4	4	2	1
11	Model most robust to many-shot demos	1	1	5	5
12	Ordering with highest compliance	4	4	5	5
13	Framework adapted for TTS (cross-attention)	2	2	5	5
14	Token category with lowest variance	3	0	0	0
15-20	Not completed (run stopped by API rate limit)	0	0	0	0

5. Advanced technique and its limits

The advanced technique in this pipeline is MMR-based re-ranking over an oversampled candidate pool (fetch_k=10, returning the top 3 after diversity re-ranking), compared against plain cosine similarity as the baseline. I had originally planned to add a cross-encoder re-ranker on top of this pulling back a larger candidate set with FAISS and then re-scoring with a model like ms-marco-MiniLM, which tends to catch relevance mismatches that bi-encoder embeddings miss. I did not get this fully working in time to include results for it, which I would rather admit plainly than paper over: the cross-encoder added noticeable latency per question on top of an already slow Groq free-tier loop, and debugging the score normalisation between FAISS distances and cross-encoder logits took longer than the time I had left. That is a real gap in this submission, not a design choice I would defend.

What the MMR experiment did show clearly is that diversity re-ranking is not free. It helps when the top cosine-similarity hits are redundant copies of the same passage, which happened often in the small-chunk index because 500-character chunks around a heading tend to repeat context. It does not help, and can slightly hurt, when the top hits are already diverse but incomplete, because swapping one relevant chunk for a moderately-relevant-but-different one just trades one gap for another.

6. Failure analysis

The single biggest failure in this project was operational rather than algorithmic: the evaluation run against the Groq free tier stopped partway through question 14 out of 20, because the API key hit its rate limit after roughly an hour of continuous requests (each question makes five calls, so twenty questions is a hundred completions). The code has a fixed two-second sleep between calls, which is not adaptive to the actual rate limit response, so the run simply stalled and had to be restarted. The output log in this repository is the partial run; the questions after 14 have never been scored. If I were doing this again I would add exponential backoff on 429 responses and checkpoint results to disk after every question, rather than holding everything in memory and printing at the end, so a rate-limit failure loses minutes of work rather than the whole run.

The second failure worth naming is that the Groq API key is hard-coded directly into main.py rather than read from an environment variable. It works for a personal experiment but it is not something I would ship, and the key has since been rotated. This is the kind of mistake that is easy to make when you are focused on getting a pipeline running end to end and easy to forget to clean up afterwards, which is exactly why it belongs in a failure section rather than getting quietly fixed and left unmentioned.

A more interesting failure is the split-fact problem described in section 3: small chunking created answers that were confidently wrong-by-omission, saying 'not enough information' when the information was present in the corpus but scattered across a chunk boundary. That is a harder problem than it looks, because increasing chunk size just moves the boundary rather than removing it, and the real fix semantic or sentence-aware chunking that respects where an idea actually starts and ends, rather than a fixed character count is exactly the kind of chunking strategy I did not have time to also test as a third configuration.

7. Scaling to 10,000 documents

Everything about this pipeline was sized for five documents, and several choices would need to change well before document 10,000. FAISS in memory works fine for fifty pages; at ten thousand documents the index would need to move to a persistent, sharded vector store such as Qdrant or Milvus, with metadata

filtering so a query can be scoped to a document type or date range before similarity search even runs, rather than searching the whole corpus every time.

Chunking would also need to stop being a single global choice. A fixed 500 or 1500 character split works when every document is similar in structure; across ten thousand documents from different sources, a chunking strategy that is aware of document structure — headings, tables, code blocks — would matter far more than it did here, where the corpus was uniform enough that a character-based splitter was good enough.

On the generation side, a single Groq API key on the free tier would not survive real production traffic, and the two-second sleep-based rate limiting in this code is a prototype hack, not a strategy. Production would need a proper request queue with backoff, batching where the provider supports it, and almost certainly a paid tier or a self-hosted model for cost control at that volume. I would also add automated evaluation rather than manually scoring twenty questions by eye — something like RAGAS metrics (faithfulness, answer relevance, context precision) run on a held-out question set after every pipeline change, so a chunking or retrieval change can be validated in minutes instead of by rereading a hundred model outputs.

Learning diary

The hardest engineering decision in this project was not which chunk size to pick — it was deciding, honestly, that the cross-encoder re-ranker I wanted to build was not going to make it into this submission in a state I could stand behind. It would have been easy to include a half-working version with cherry-picked results, and it was tempting, because it is the piece of the assignment that looks most like 'advanced technique'. I chose instead to keep MMR as the second retrieval configuration and to write plainly about the gap, because a report that quietly hides an unfinished feature teaches me less than one that says exactly where the time ran out and why.

The other lesson came from the rate-limit failure. My first instinct on seeing the run stop at question 14 was to just rerun it and hope it would finish this time. It did not occur to me until afterwards that the actual engineering lesson was one level up: a pipeline that cannot survive a rate-limit error without losing an hour of work is not a robust pipeline, no matter how good the retrieval logic underneath it is. Small chunk sizes, MMR versus cosine, embedding model choice — all of that matters far less than whether the system checkpoints its own progress. That is the kind of thing that is obvious in hindsight and invisible until you have watched a run die at question 14 of 20.